

Integración de seguridad en pipelines DevOps para prevenir vulnerabilidades en APIs web: una revisión sistemática de la literatura

Miguel Angel Davila Hernandez
Instituto Tecnológico de Matamoros
Matamoros, México
L23260173@matamoros.tecnm.mx

Resumen—La diversidad de herramientas y estrategias para incorporar controles de seguridad de APIs en los procesos DevOps dificulta que las organizaciones determinen cuáles ofrecen resultados efectivos sin afectar la velocidad de despliegue. El objetivo de esta revisión es identificar, evaluar críticamente y sintetizar la evidencia científica y empírica disponible sobre las mejores prácticas para integrar la seguridad de APIs en pipelines DevOps, y determinar su impacto en la reducción de vulnerabilidades de software. El proceso de identificación y selección se documentó conforme a PRISMA 2020 y consideró IEEE Xplore, ACM Digital Library, literatura gris y *snowballing*, con estudios publicados entre 2020 y 2026. Después de evaluar 108 registros mediante tres etapas de selección, se incluyeron 24 fuentes en la síntesis final. Los resultados muestran una integración por capas de SAST, DAST, SCA, pruebas de APIs y validación de IaC; sin embargo, la evidencia cuantitativa de reducción se concentra en configuraciones y políticas, mientras que las vulnerabilidades funcionales de APIs se evalúan principalmente mediante métricas de detección. Se concluye que la automatización favorece la detección temprana y reduce trabajo manual, pero aún no existe una combinación universal de herramientas ni evidencia uniforme para cuantificar la reducción de cada categoría de vulnerabilidad.

Index Terms—seguridad de APIs, DevSecOps, DevOps, vulnerabilidades de software, pruebas de seguridad automatizadas, revisión sistemática de la literatura.

I. INTRODUCCIÓN

Las interfaces de programación de aplicaciones (APIs) web constituyen un componente central de los sistemas de software modernos y permiten la comunicación entre servicios, aplicaciones y plataformas. Esta función también amplía la superficie de ataque, como refleja la clasificación de riesgos de seguridad para APIs de OWASP [2], y exige que la seguridad se considere durante todo el ciclo de desarrollo, en lugar de tratarse únicamente como una verificación previa al despliegue. En este contexto, DevSecOps propone integrar controles y pruebas de seguridad automatizadas en los pipelines DevOps con el propósito de detectar y mitigar vulnerabilidades de manera temprana [1].

Sin embargo, el conocimiento técnico y metodológico acerca de esta integración se encuentra fragmentado. Aunque existen múltiples herramientas comerciales y de código abierto para proteger APIs, las organizaciones no cuentan con un marco consolidado que permita comparar la efectividad y eficiencia de las estrategias DevSecOps, su capacidad para

reducir vulnerabilidades y su efecto sobre la velocidad de despliegue. Asimismo, parte de la evidencia se encuentra en estudios científicos indexados y otra parte en literatura gris, como marcos técnicos e informes del sector. Esta dispersión dificulta la selección informada de prácticas, herramientas y controles de seguridad.

El objetivo de esta revisión de la literatura sistemática es identificar, evaluar críticamente y sintetizar la evidencia científica y empírica disponible sobre las mejores prácticas para integrar la seguridad de APIs en pipelines DevOps, y determinar su impacto real en la reducción de vulnerabilidades de software.

La revisión persigue tres contribuciones principales. Primero, busca reducir la brecha entre academia e industria mediante la integración de estudios empíricos indexados y literatura gris relevante, a fin de ofrecer una visión actualizada y aplicable frente a amenazas emergentes. Segundo, pretende apoyar la toma de decisiones de arquitectos de software y responsables de ciberseguridad mediante la identificación del impacto medible de la automatización de pruebas de seguridad y de sus posibles efectos en el pipeline de desarrollo. Tercero, proporcionará sustento científico para posteriores actividades prácticas de análisis y mitigación de vulnerabilidades, de manera que las propuestas resultantes estén alineadas con el estado del arte.

El resto del artículo se organiza de la siguiente manera: la Sección II presenta los antecedentes; la Sección III describe el protocolo; las Secciones IV y V exponen y discuten los hallazgos; finalmente, las Secciones VI y VII presentan las amenazas a la validez y las conclusiones.

II. ANTECEDENTES Y TRABAJOS RELACIONADOS

II-A. Fundamentos conceptuales

Una API web expone operaciones y datos mediante interfaces consumibles a través de protocolos de red. En arquitecturas modernas, estas interfaces suelen implementarse como servicios REST, GraphQL o microservicios distribuidos. La separación entre servicios favorece la escalabilidad y el despliegue independiente, pero también amplía la superficie de ataque, multiplica las identidades técnicas y dificulta observar y controlar la comunicación interna [22], [23].

DevOps integra desarrollo y operaciones mediante automatización, entrega continua y retroalimentación rápida. DevSecOps amplía este enfoque al incorporar la seguridad como una responsabilidad continua dentro del ciclo de vida, en lugar de reservarla para una revisión final [1], [16]. En un pipeline CI/CD, esta integración puede abarcar requisitos de seguridad, análisis de código, comprobación de dependencias, pruebas automatizadas, validación de infraestructura, pasarelas de seguridad, monitoreo y gestión de hallazgos.

Los controles se clasifican según el momento y el objeto analizado. SAST inspecciona código o configuraciones sin ejecutar la aplicación; DAST evalúa el comportamiento de un sistema desplegado; IAST combina información de ejecución con instrumentación interna; y SCA identifica riesgos en dependencias y componentes de terceros. En entornos cloud-native se añaden el análisis de infraestructura como código (IaC), Policy as Code, el escaneo de imágenes y la validación de configuraciones de contenedores y orquestadores [6], [16], [18].

Las vulnerabilidades de APIs no se limitan a defectos sintácticos. OWASP API Security Top 10 incluye riesgos de autorización a nivel de objeto o función, autenticación, exposición de propiedades y configuraciones incorrectas [2]. En microservicios, estas categorías se relacionan además con identidades entre servicios, robo de tokens, privilegios excesivos, comunicación este-oeste y falta de visibilidad distribuida [13], [22], [23].

Esta revisión combina literatura académica con literatura gris. Esta última incluye guías, estándares e informes técnicos que permiten observar prácticas y herramientas de la industria que pueden no estar documentadas en publicaciones revisadas por pares. La combinación resulta pertinente en DevSecOps, donde la evolución de plataformas y controles suele avanzar con mayor rapidez que su evaluación académica [5], [16], [23].

II-B. Revisiones y estudios relacionados

Los trabajos relacionados pueden agruparse en tres líneas. La primera estudia métricas y modelos de amenazas. P01 realizó una revisión multivocal de 92 fuentes publicadas entre 2018 y 2023 y propuso 12 métricas cuantitativas para microservicios web desarrollados con DevSecOps [5]. P02 examinó modelos de amenazas y propuso una arquitectura de tres capas: fundacional, integración DevSecOps y automatización/orquestación. Su alcance se orientó a entornos multi-nube y dejó abierta la integración de Zero Trust, predicción de amenazas y respuesta automatizada [6].

La segunda línea aborda los pipelines y la adopción de DevSecOps. P09 revisó 59 estudios sobre autenticación y autorización en CI/CD cloud-native, identificó siete clases de vulnerabilidades y relacionó defensas como OAuth 2.0, mTLS y RBAC con los riesgos tratados [13]. Su énfasis académico deja espacio para validar los mecanismos en entornos industriales y considerar herramientas propietarias. P12 analizó 53 estudios de 2019 a 2025 e identificó barreras, ventajas y estrategias de implementación; concluyó que la adopción

depende tanto de la automatización como de la cultura, las competencias y los recursos de la organización [16].

La tercera línea se concentra en microservicios. SB1 examinó 54 publicaciones de 2015 a 2023 y encontró que la literatura presta mayor atención a las mitigaciones que a la caracterización de amenazas, con vacíos en autenticación y comunicación entre servicios y en la correspondencia explícita entre amenazas y controles [22]. SB2 analizó 57 fuentes de literatura gris, seleccionadas de 312 documentos publicados entre enero de 2011 y marzo de 2021. Este estudio destacó problemas de confianza, superficie de ataque, contenedores y visibilidad, además de discrepancias entre recomendaciones académicas y prácticas industriales [23].

La presente RLS se diferencia al conectar cuatro dimensiones en un mismo protocolo: herramientas de seguridad para APIs, estructura de los workflows, métricas de impacto y evidencia de reducción por clase de vulnerabilidad. Además, integra estudios académicos, literatura gris y snowballing, y distingue entre reducción medida, mejoras operativas y mitigaciones únicamente propuestas. Su alcance se limita principalmente a APIs en entornos cloud-native, microservicios y CI/CD, por lo que no pretende generalizar automáticamente los resultados a toda API web.

III. METODOLOGÍA

La identificación, el cribado, la evaluación de elegibilidad y el reporte de la selección se organizaron conforme a la declaración PRISMA 2020 [25]. El protocolo se definió antes de ejecutar la búsqueda para favorecer la transparencia, la trazabilidad y la reproducibilidad del proceso.

III-A. Objetivo y preguntas de investigación

El objetivo general es identificar, evaluar críticamente y sintetizar la evidencia científica y empírica sobre las mejores prácticas de integración de seguridad de APIs en pipelines DevOps, determinando su impacto en la reducción de vulnerabilidades de software y considerando su relación con la eficiencia y la velocidad de despliegue.

La pregunta general que guía la revisión es: *¿Cuál es el impacto de la integración de prácticas de seguridad automatizadas en los pipelines de DevOps/DevSecOps sobre la reducción de vulnerabilidades durante el desarrollo de APIs web en la industria de software actual?*

III-B. Clasificación y trazabilidad de las fuentes

Para conservar la trazabilidad durante la selección, extracción y síntesis, cada fuente recibió un identificador estable según su procedencia. El prefijo LG corresponde a literatura gris; P, a estudios científicos recuperados mediante la búsqueda primaria; y SB, a estudios científicos incorporados mediante snowballing. Se empleó una numeración consecutiva de dos dígitos para LG y P, mientras que los registros de snowballing se identificaron como SB1–SB3. El corpus preliminar contiene 24 fuentes: cuatro de literatura gris, 17 de la búsqueda primaria y tres obtenidas mediante snowballing, como resume la Tabla II.

Cuadro I
COMPARACIÓN DE LAS REVISIONES RELACIONADAS

Estudio	Tipo y fuentes	Periodo / N	Contribución principal	Vacío respecto de la presente RLS
P01	Revisión multivocal; IEEE Xplore, Springer y Google Scholar	2018–2023 / 92	Doce métricas cuantitativas para microservicios web y DevSecOps.	Propone métricas, pero no integra una evaluación específica de herramientas, workflows y reducción de vulnerabilidades de APIs.
P02	Revisión de modelos de amenazas	No informado	Arquitectura de tres capas para DevSecOps multi-nube.	Requiere adaptar los modelos a entornos dinámicos e incorporar Zero Trust, predicción y respuesta automatizada.
P09	RLS; IEEE, Scopus, ACM, ScienceDirect y Springer-Link	No informado / 59	Taxonomía de siete vulnerabilidades de identidad y mapeo de defensas en CI/CD cloud-native.	Predomina la evidencia académica y faltan validaciones empíricas y un marco estandarizado de control de acceso.
P12	RLS cualitativa; ACM, IEEE, Springer y ScienceDirect	2019–2025 / 53	Barreras, ventajas y estrategias para implementar DevSecOps.	Es generalista, no se centra en APIs y solicita evidencia industrial, métodos mixtos y literatura gris.
SB1	RLS; IEEE, SpringerLink, ScienceDirect, Wiley y ACM	2015–2023 / 54	Amenazas y mitigaciones entre servicios en micro-servicios.	Faltan taxonomías estandarizadas y una relación directa y evaluada entre cada amenaza y su mitigación.
SB2	Estudio SoK de literatura gris mediante Google	2011–2021 / 57 de 312	Perspectiva profesional sobre confianza, contenidos, visibilidad y prácticas de seguridad.	Persisten brechas academia-industria y poca evidencia sobre arquitecturas políglotas, contenidos inmutables, mínimo privilegio y rate limiting.

Cuadro II
CLASIFICACIÓN PRELIMINAR DE LAS FUENTES

Procedencia	Identificadores	Cantidad
Literatura gris	LG01–LG04	4
Búsqueda primaria	P01–P17	17
<i>Snowballing</i>	SB1–SB3	3
Total		24

Cuadro III
PREGUNTAS DE INVESTIGACIÓN

ID	Pregunta de investigación
RQ1	¿Cuáles son las herramientas de análisis de seguridad (SAST, DAST y SCA) y las metodologías de prueba más utilizadas en la literatura para proteger APIs web dentro de pipelines de CI/CD?
RQ2	¿Cómo se estructuran y configuran los flujos de trabajo (<i>workflows</i>) para automatizar los controles de seguridad de APIs sin degradar la velocidad de despliegue en DevOps?
RQ3	¿Qué métricas e indicadores de impacto, como la tasa de defectos, el tiempo de remediación y las vulnerabilidades remanentes en producción, reporta la literatura tras adoptar un enfoque DevSecOps en APIs?
RQ4	¿Cuáles son las vulnerabilidades específicas de APIs que presentan una mayor tasa de reducción mediante estas prácticas automatizadas?

III-C. Formulación mediante PICOC

III-D. Fuentes de información y estrategia de búsqueda

La identificación de fuentes se realizó entre el 16 y el 27 de junio de 2026, de acuerdo con el cronograma de la revisión, mediante tres vías complementarias. Primero, se ejecutó una búsqueda académica en IEEE Xplore y ACM Digital Library. Segundo, se efectuó una búsqueda dirigida de literatura gris en OWASP y otras fuentes técnicas relevantes. Tercero, el conjunto de estudios se amplió mediante *snowballing*. Se consideraron publicaciones comprendidas entre 2020 y 2026.

La cadena se construyó a partir de tres bloques conceptuales unidos mediante el operador AND. Dentro de cada bloque, los

Cuadro IV
COMPONENTES PICOC DE LA REVISIÓN

Componente	Descripción
Población	APIs web (REST, GraphQL y microservicios) y sus componentes de desarrollo.
Intervención	Integración de prácticas y herramientas de seguridad (SAST, DAST, SCA y pruebas de contrato) en pipelines DevOps/DevSecOps.
Comparación	Desarrollo tradicional, con seguridad al final del ciclo o pruebas de penetración aisladas, y ausencia de automatización de seguridad.
Resultados	Reducción de vulnerabilidades de seguridad, especialmente las contempladas en OWASP API Security Top 10; eficiencia de la detección temprana y mejora de la integridad de los datos.
Contexto	Entornos modernos de desarrollo de software, ciclos de vida de desarrollo automatizados e ingeniería de software basada en la nube.

términos equivalentes se conectaron mediante OR, como se detalla en la Tabla V. Esta separación permite verificar los paréntesis y reproducir la lógica sin depender del ajuste visual de una línea extensa.

La sintaxis se adaptó a las reglas de IEEE Xplore y ACM Digital Library, conservando los tres bloques, sus paréntesis y la relación booleana. Para literatura gris se utilizaron los mismos conceptos como consultas dirigidas. Se admitieron documentos en inglés y español, limitados a artículos de revista, trabajos de conferencia revisados por pares y reportes oficiales de literatura gris.

III-E. Criterios de selección

Estos criterios buscaron asegurar un nivel mínimo de rigor de las fuentes, mantener la actualidad tecnológica del corpus y garantizar su correspondencia con la pregunta de investigación. La disponibilidad del texto completo fue necesaria para la extracción de datos, mientras que la eliminación de duplicados evitó contabilizar un mismo estudio más de una vez.

Cuadro V
DESCOMPOSICIÓN REPRODUCIBLE DE LA CADENA DE BÚSQUEDA

Bloque	Dimensión	Términos unidos mediante OR
B1	API o arquitectura	API, Application Programming Interface, Web API, REST, GraphQL, microservices
B2	Entorno de desarrollo	DevOps, DevSecOps, CI/CD, pipeline*, continuous integration
B3	Seguridad e impacto	secur*, vulnerab*, mitigat*, reduction, AppSec, OWASP

Expresión exacta: B1 AND B2 AND B3. Los saltos siguientes son únicamente visuales.

```
("API" OR "Application Programming Interface" OR "Web API" OR "REST" OR "GraphQL" OR "microservices")
AND
("DevOps" OR "DevSecOps" OR "CI/CD" OR "pipeline*" OR "continuous integration")
AND
("secur*" OR "vulnerab*" OR "mitigat*" OR "reduction" OR "AppSec" OR "OWASP")
```

Cuadro VI
CRITERIOS DE INCLUSIÓN Y EXCLUSIÓN

Dimensión	Criterio de inclusión	Criterio de exclusión
Tipo de publicación	CI1: Artículos de revistas, conferencias revisadas por pares o reportes oficiales de literatura gris, como OWASP y NIST.	CE1: Blogs, hilos de foros, opiniones comerciales, presentaciones breves o tesis no publicadas.
Año de publicación	CI2: Documentos publicados entre 2020 y 2026.	CE2: Documentos publicados antes de 2020.
Idioma	CI3: Documentos en inglés o español.	CE3: Documentos escritos en cualquier otro idioma.
Temática principal	CI4: Estudios que mencionen explícitamente la seguridad de APIs REST, GraphQL o microservicios y su relación con entornos DevOps o CI/CD.	CE4: Estudios sobre seguridad general de redes, criptografía pura o desarrollo de software sin mención de APIs.
Acceso al documento	CI5: Texto completo disponible mediante acceso directo o institucional.	CE5: Documentos de los que únicamente se encuentre disponible el resumen.
Duplicados	CI6: Registro original o versión más completa localizada durante las búsquedas.	CE6: Registros repetidos del mismo estudio en distintas bases de datos.

III-F. Proceso de selección

La búsqueda produjo inicialmente 108 registros. La selección se organizó en tres etapas. En la etapa 1 se aplicaron el intervalo de publicación de 2020 a 2026 (CI2), los idiomas inglés o español (CI3) y la eliminación de duplicados (CE6), con lo cual permanecieron 95 registros. De los 13 registros retirados antes del cribado, cinco correspondieron a duplicados y ocho a otros motivos relacionados con los filtros de año e idioma. En la etapa 2 se examinaron el título y el resumen de acuerdo con la pertinencia temática (CI4), el tipo de publicación no deseado (CE1) y la temática fuera de contexto (CE4); esta evaluación redujo el conjunto a 59 documentos. En la etapa 3 se verificó el acceso al texto completo (CI5) y se confirmó la elegibilidad mediante la lectura integral. Cinco documentos no pudieron recuperarse y otros 30 fueron excluidos durante la evaluación de texto completo: 18 por tratar seguridad general sin un enfoque específico en APIs (CE4) y 12 por corresponder a materiales no elegibles según el tipo de publicación (CE1). El corpus final quedó compuesto por 24 fuentes. Todas las etapas de cribado, recuperación y evaluación fueron realizadas por un único revisor.

No se identificaron incompatibilidades relevantes en la sintaxis ni en los operadores de la cadena de búsqueda. La principal dificultad fue operativa: IEEE Xplore y ACM Digital Library limitaron la exportación masiva o automatizada de resultados y, al no disponer de credenciales asociadas con una suscripción institucional, los textos completos tuvieron que recuperarse manualmente y de forma individual. Este procedimiento incrementó el tiempo de recolección y contribuyó

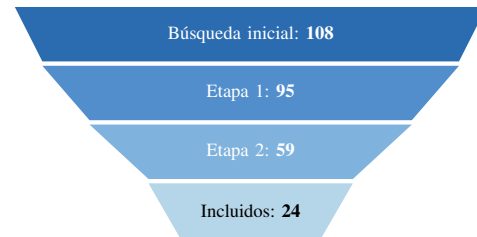


Figura 1. Resumen complementario de los registros conservados por etapa.

a que cinco documentos no pudieran recuperarse, aunque los 24 registros finalmente incluidos sí fueron examinados a texto completo.

La Figura 1 ofrece un resumen visual complementario del número de registros que permaneció en cada etapa: 108 en la búsqueda inicial, 95 después de la etapa 1, 59 después de la etapa 2 y 24 en el corpus final. Su propósito es facilitar la comparación de la reducción acumulada; el desglose formal de exclusiones, documentos no recuperados y estudios incluidos se presenta exclusivamente en el diagrama PRISMA de la Figura 2.

III-G. Snowballing

El proceso complementario de *snowballing* siguió una estrategia exclusivamente hacia atrás (*backward snowballing*). Se revisaron manualmente y de forma exhaustiva las listas de referencias bibliográficas de estudios secundarios clave previamente seleccionados, con el propósito de localizar artículos

Cuadro VII
RESULTADOS DEL PROCESO DE SELECCIÓN POR FUENTE DE INFORMACIÓN

Fuente	Búsqueda inicial	Etapla 1	Etapla 2	Etapla 3
IEEE Xplore	61	53	31	7
ACM Digital Library	29	27	19	10
OWASP.org	3	3	3	3
NIST.gov	7	6	0	0
Salt Security	1	1	1	1
<i>Snowballing</i> (búsqueda secundaria)	7	5	5	3
Total	108	95	59	24

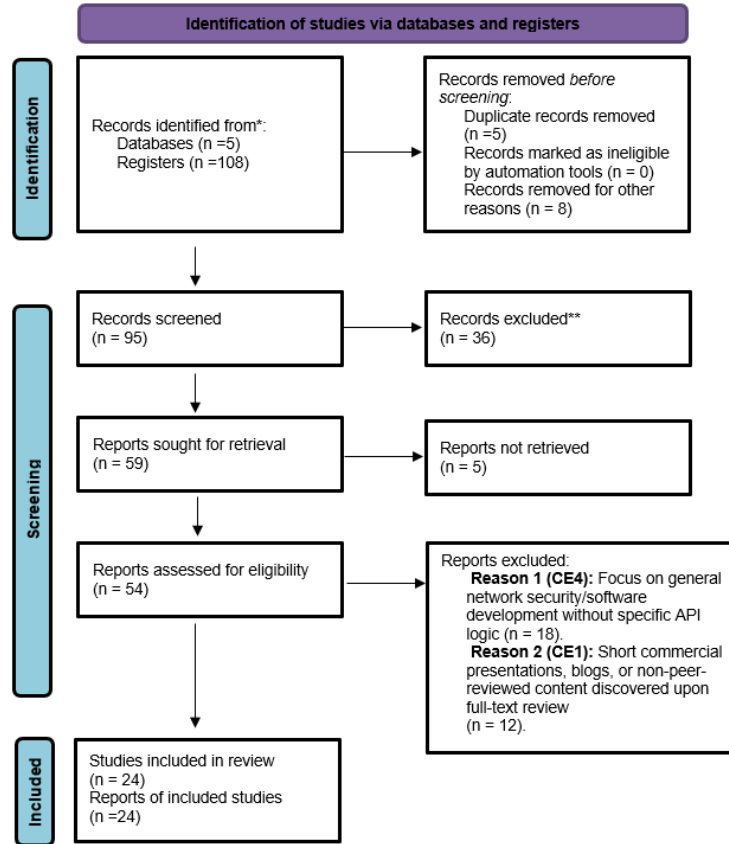


Figura 2. Diagrama PRISMA del proceso de identificación y selección de fuentes.

científicos relevantes que pudieran haber sido omitidos por los términos y filtros de la búsqueda primaria.

El rastreo se desarrolló en dos niveles. El primer nivel partió de P09, *Systematic Review of Identity-Centric Security in Cloud-Native CI/CD Pipelines* [13], recuperado durante la búsqueda primaria. La revisión de sus referencias permitió identificar e incluir SB1, una revisión sistemática sobre amenazas y estrategias de mitigación entre servicios en arquitecturas de microservicios [22], y SB2, un estudio de conocimiento sistematizado sobre los retos y las buenas prácticas de seguridad en aplicaciones de microservicios [23].

En el segundo nivel se examinaron las referencias de SB2. Esta iteración produjo dos artículos candidatos; uno fue des-

cartado después de aplicar los criterios de elegibilidad y el otro se incorporó como SB3, *Sidecar-Based Path-Aware Security for Microservices* [24]. En conjunto, la búsqueda secundaria produjo siete registros: cinco superaron las primeras etapas de cribado y tres fueron incorporados al corpus final como SB1–SB3.

III-H. Evaluación de calidad

No se aplicó una escala de Sí/Parcial/No, un cuestionario de puntuación ni un umbral numérico de calidad. La admisibilidad metodológica se determinó mediante la superación sucesiva de los criterios de inclusión y exclusión documentados en el flujo PRISMA: actualidad de la publicación, idioma, tipo y procedencia de la fuente, pertinencia temática,

disponibilidad del texto completo y eliminación de duplicados. En consecuencia, este procedimiento funcionó como un control de elegibilidad del corpus y no como una evaluación independiente del riesgo de sesgo o de la calidad interna de cada estudio.

III-I. Extracción y síntesis de datos

La extracción se realizó manualmente mediante una matriz estructurada, conservando para cada registro su identificador LG, P o SB. Las variables se agruparon según su relación con las cuatro preguntas de investigación, como se muestra en la Tabla VIII.

El formulario estandarizado se aplicó individualmente a los 17 estudios primarios (P01–P17), las cuatro fuentes de literatura gris (LG01–LG04) y los tres estudios obtenidos por snowballing (SB1–SB3). Cada hallazgo se vinculó con RQ1–RQ4 para mantener criterios uniformes al registrar herramientas, configuración de pipelines, vulnerabilidades, mitigaciones y métricas de impacto.

Como comprobación auxiliar se utilizó Consensus, restringiendo las consultas al corpus seleccionado. La herramienta permitió contrastar los datos registrados en la matriz con los documentos originales y localizar posibles diferencias de interpretación. Cuando surgió alguna discrepancia, la decisión final se tomó mediante una nueva revisión manual de la fuente; por tanto, la salida de la herramienta no se consideró evidencia independiente ni sustituyó el juicio del investigador.

Los datos se sintetizaron mediante una síntesis narrativa y una agrupación temática orientada por RQ1–RQ4. Debido a que la revisión fue desarrollada por un único investigador, la extracción y codificación no fueron contrastadas mediante acuerdo entre revisores; la consistencia se apoyó en el formulario estandarizado, la comprobación auxiliar y el uso de identificadores estables durante todo el análisis.

IV. RESULTADOS

IV-A. Descripción de los estudios incluidos

El corpus final quedó compuesto por 24 fuentes: cuatro documentos de literatura gris, 17 estudios recuperados mediante la búsqueda primaria y tres estudios incorporados por *backward snowballing*. Las publicaciones académicas incluyeron artículos de revista y trabajos de conferencia publicados entre 2022 y 2026. Temáticamente, el corpus abarcó seguridad de APIs REST y GraphQL, arquitecturas de microservicios, automatización DevSecOps, seguridad de contenedores, infraestructura como código, observabilidad y métricas de seguridad.

IV-B. Respuesta a la RQ1: herramientas y metodologías

El análisis permitió organizar las tecnologías en tres grupos: pruebas estáticas de seguridad de aplicaciones (SAST), pruebas dinámicas o interactivas y pruebas específicas de APIs (DAST/IAST), y herramientas para análisis de composición, contenedores e infraestructura como código (SCA/IaC). Algunos estudios nombraron productos concretos, mientras que otros analizaron la categoría de control sin asociarla con una implementación específica [6], [8], [16].

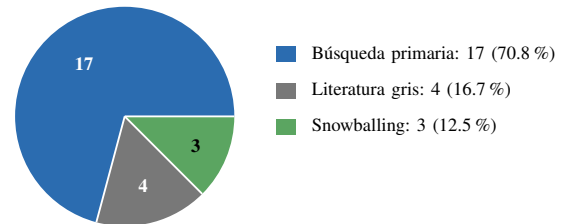


Figura 3. Distribución de las 24 fuentes según su procedencia.

IV-B1. Pruebas estáticas de seguridad: SAST se presenta como un control temprano para inspeccionar código, componentes y configuraciones antes del despliegue. SonarQube fue documentado dentro de una solución automatizada para CI/CD [17], mientras que Mega-Linter y Super-Linter aparecieron como mecanismos de inspección continua integrables en repositorios y flujos de GitHub [18]. SB2 también registró OWASP Source Code Analysis como referencia para el escaneo de código [23]. La inspección estática se extendió a IaC mediante herramientas como IaC Scan Runner y VAT [6], [18].

IV-B2. DAST, IAST y pruebas de APIs: La literatura describió IAST principalmente como una categoría de control, sin una herramienta dominante [6], [16]. En DAST, SB2 identificó OWASP ZAP, HP Fortify y Neuralegion NexDAST para la evaluación de aplicaciones y microservicios en ejecución [23]. SecCo, por su parte, orquestó técnicas de *Web Application Security Testing*, *Security API Scanning* y *Behaviour Driven Security Testing* sin depender de un único producto DAST [17].

Las pruebas específicas de APIs mostraron mayor especialización. Para REST se evaluaron RESTest, Schemathesis, RESTler, RestTestGen y EvoMaster [15]. EvoMaster también fue estudiado en APIs GraphQL mediante modalidades de caja blanca y caja negra [21]. Asimismo, se reportaron módulos experimentales como APISecurityValidator y SecurityTestRunner para automatizar fuzzing y validaciones basadas en OpenAPI [8].

IV-B3. SCA, contenedores e infraestructura como código: Snyk fue la solución SCA más recurrente en la evidencia proporcionada, asociada con la comprobación continua de dependencias y componentes de terceros [12], [16], [18]. Para imágenes de contenedores se identificaron Clair y Anchore, mientras que Docker Bench y CIS Docker Scanner se emplearon para validar configuraciones [23]. SB1 documentó además CAVAS como prototipo para detectar y validar vulnerabilidades en imágenes y entornos de microservicios [22].

Kubernetes, OpenShift, Istio y Azure Kubernetes Service aparecieron como plataformas sobre las que se integran controles de seguridad [8], [9], [23]. La protección de secretos y certificados incluyó HashiCorp Vault, Spring Vault, Certbot y WildFly Elytron [12], [23]. Finalmente, el bloque IaC/PaC reunió Terraform, Ansible, Tfsec, Open Policy Agent, Kyverno, Checkov, KICS, KubeLinter, Terrascan y Datree [8], [9], [12], [16], [18].

Cuadro VIII
CATEGORÍAS EMPLEADAS PARA LA EXTRACCIÓN DE DATOS

Categoría	Datos extraídos	RQ
Vulnerabilidades de APIs	Brechas y amenazas vinculadas con la lógica de negocio y la exposición de datos en APIs REST, GraphQL y microservicios, diferenciadas de vulnerabilidades perimetrales o generales de red.	RQ4
Herramientas y automatización	Adopción e integración de SAST, DAST, IAST, análisis de composición de software y otras metodologías de prueba de seguridad dentro del ciclo de vida.	RQ1
Configuración del pipeline DevSecOps	Estructura de los mecanismos CI/CD y forma en que los controles se incorporan como código (<i>Security as Code</i>) en las etapas del flujo.	RQ2
Mitigaciones y resultados	Estrategias arquitectónicas, como mallas de servicios y patrones <i>sidecar</i> , junto con métricas, resultados empíricos y evidencia de reducción de vulnerabilidades.	RQ3–RQ4

Cuadro IX
HERRAMIENTAS Y ENFOQUES SAST IDENTIFICADOS

Herramienta o enfoque	Función reportada	Evidencia
SonarQube	Análisis centralizado de código integrado en soluciones de seguridad para CI/CD.	[17]
Mega-Linter y Super-Linter	Inspección continua y ejecución como workflows asociados al repositorio.	[18]
OWASP Source Code Analysis	Escaneo estático de código mencionado en el contexto de microservicios.	[23]
IaC Scan Runner y VAT	Inspección estática de scripts y configuraciones de infraestructura.	[6], [18]
SAST genérico	Control estándar dentro de pipelines DevSecOps, sin producto específico.	[6], [8], [16]

Cuadro X
HERRAMIENTAS DAST, IAST Y DE PRUEBAS DE APIs

Categoría	Herramientas o técnicas	Evidencia
DAST	OWASP ZAP, HP Fortify y Neuralegion NexDAST.	[23]
IAST	Categoría descrita conceptualmente, sin predominio de productos concretos.	[6], [16]
Orquestación de pruebas	Web Application Security Testing, Security API Scanning y Behaviour Driven Security Testing.	[17]
Pruebas REST	RESTTest, EvoMaster, Schemathesis, RESTler y RestTestGen.	[15]
Pruebas GraphQL	EvoMaster en modalidades de caja blanca y caja negra.	[21]
Fuzzing basado en OpenAPI	APISecurityValidator y SecurityTestRunner.	[8]

Cuadro XI
HERRAMIENTAS SCA, DE CONTENEDORES Y DE INFRAESTRUCTURA COMO CÓDIGO

Categoría	Herramientas identificadas	Evidencia
SCA y dependencias	Snyk y comprobación de componentes de ecosistemas como npm.	[12], [16], [18]
Imágenes y contenedores	Clair, Anchore, Docker Bench, CIS Docker Scanner y CAVAS.	[22], [23]
Plataformas de orquestación	Kubernetes, OpenShift, Istio y Azure Kubernetes Service.	[8], [9], [23]
Secretos y certificados	HashiCorp Vault, Spring Vault, Certbot y WildFly Elytron.	[12], [23]
IaC y Policy as Code	Terraform, Ansible, Tfsec, OPA, Kyverno, Checkov, KICS, KubeLinter, Terrascan y Datree.	[8], [9], [12], [16], [18]

En síntesis, la evidencia no muestra una herramienta universalmente dominante. En cambio, respalda una combinación por capas: SAST durante la construcción, SCA e IaC antes del despliegue, pruebas DAST y específicas de APIs en entornos ejecutables, y controles de contenedores, secretos y políticas en la plataforma de operación.

La Figura 4 contabiliza tecnologías con nombre propio una sola vez, aunque aparezcan en varios estudios o se utilicen en más de un tipo de prueba. No se incluyeron categorías genéricas como SAST o IAST ni técnicas sin una implementación nombrada. El bloque SCA/IaC incluye las herramientas de dependencias, contenedores, orquestación, secretos y políticas

descritas en la Tabla XI.

IV-C. Respuesta a la RQ2: automatización de los workflows

La evidencia no define un pipeline universal, pero sí muestra una secuencia recurrente que combina *shift-left* con controles durante la ejecución [12], [17], [19]. Esta secuencia comienza con requisitos y propiedades de seguridad, continúa con análisis estáticos antes de liberar los artefactos y concluye con pruebas dinámicas, monitoreo y retroalimentación hacia desarrollo.

IV-C1. Orden de ejecución de los controles: En diseño y planificación se especifican requisitos, amenazas y propiedades

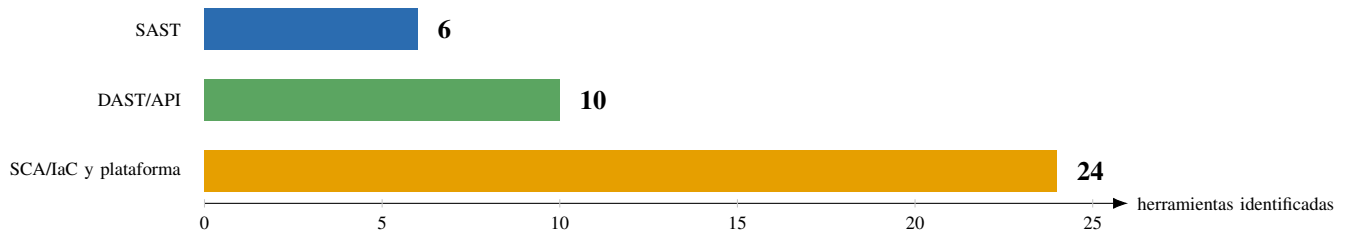


Figura 4. Cantidad de herramientas o tecnologías identificadas por categoría funcional.

verificables antes de iniciar la entrega [17], [19]. Durante la construcción y el predespliegue se ejecutan SAST, SCA, inspecciones de IaC y validaciones de políticas, de modo que el pipeline pueda detener artefactos que incumplan las reglas definidas [8], [17], [18]. Después del despliegue se incorporan DAST, monitoreo, detección de anomalías y mecanismos de realimentación para cubrir defectos que requieren contexto de ejecución [12], [17], [18], [20].

El estudio sobre Helm presentó uno de los flujos más secuenciados: análisis estático inicial, eliminación o mitigación de violaciones, prueba funcional del chart, regeneración del artefacto y reanálisis final con el mismo analizador o con uno diferente [9]. VeriDevOps propuso una lógica complementaria basada en especificar requisitos, generar monitores de trazas, localizar causas raíz e identificar fallos de código y diseño durante todo el ciclo [19].

IV-C2. Plataformas de integración y despliegue: Las implementaciones se distribuyeron entre servicios administrados, servidores de automatización y entornos nativos de nube. GitHub Actions, GitLab CI, Azure Pipelines y CircleCI aparecieron en el análisis de ecosistemas de integración continua [14]; GitHub Actions también se utilizó en flujos de análisis de configuraciones Helm [9]. Jenkins y Bamboo se emplearon para pasarelas de seguridad y validaciones de políticas o IaC en entornos multiplataforma [8]. También se reportaron pipelines sobre Kubernetes y Tekton ligados a cambios en ramas o repositorios [6].

IV-C3. Impacto operativo: Los resultados cuantitativos muestran que la automatización reduce tareas manuales, pero introduce costos de cómputo y revisión. En la validación de políticas, las revisiones manuales añadieron entre 25 y 40 minutos por configuración, mientras que las explicaciones automatizadas redujeron el tiempo de remediación en un 60 % [8]. El mismo estudio reportó una tasa inicial de falsos positivos del 10 %; los métodos de ensamble redujeron interrupciones legítimas a costa de triplicar el consumo de cómputo y alcanzar una latencia de 3.1 segundos en un experimento con 500 casos de IaC [8].

En pruebas REST, más del 70 % de las pruebas generadas no detectaron fallos, lo cual añadió sobrecarga sin producir hallazgos en esos casos [15]. En contraste, un pipeline de detección de amenazas apoyado por aprendizaje automático logró latencias inferiores a dos segundos [7]. Los analizadores de Helm presentaron resultados inconsistentes, falsos positivos y falsos negativos que todavía requirieron revisión humana [9].

A escala industrial, la fricción también estuvo asociada con la calidad de los datos, la priorización y la devolución oportuna de hallazgos al flujo ágil [20].

En conjunto, la velocidad del pipeline depende menos de una plataforma específica que de cómo se distribuyen y priorizan los controles. Las verificaciones tempranas pueden reducir la remediación tardía, pero las pruebas costosas, los falsos positivos y la gestión deficiente de hallazgos pueden convertirse en nuevos cuellos de botella.

IV-D. Respuesta a la RQ3: métricas e indicadores de impacto

La evidencia revela una diferencia entre las métricas técnicas medidas en experimentos y los indicadores de gestión propuestos a nivel conceptual. Los estudios empíricos se concentraron en cobertura, fallos, regresiones, falsos positivos y negativos, latencia, costo de ejecución, configuraciones erróneas y vulnerabilidades remanentes. En cambio, MTTR, MTTR, densidad de defectos y *defect burn rate* aparecieron principalmente como métricas recomendadas o compiladas, sin comparaciones numéricas de antes y después dentro de los estudios analizados [5], [10], [19], [20].

IV-D1. Métricas medidas empíricamente: En pruebas REST se utilizaron cobertura, proporción de fallos, fallos únicos, regresiones, APFD (*Average Percentage of Faults Detected*) y costo de ejecución [15]. Para GraphQL se midieron cobertura de líneas y capacidad de descubrimiento de fallos [21]. En configuraciones Helm, las métricas M_FOUND, M_BREAKING y M_REMAINING representaron, respectivamente, configuraciones erróneas encontradas, cambios de remediación que afectaban la funcionalidad y vulnerabilidades que permanecían después del proceso [9].

Los enfoques basados en inteligencia artificial añadieron FPR, FNR, F1-score, latencia, tasa de detección y retención de exactitud. La validación de políticas IaC también consideró atributos cualitativos auditados, como fluidez, prominencia y consistencia de las explicaciones [8]. Por su parte, la detección de amenazas en ejecución midió la tasa de ataques identificados y el tiempo de respuesta [7]. En el contexto industrial, la gestión de hallazgos se evaluó mediante la cobertura proporcionada por las herramientas, la calidad de sus datos y la capacidad para priorizar y devolver resultados al proceso de desarrollo [20].

IV-D2. Resultados cuantitativos: Las pruebas REST alcanzaron una cobertura de hasta 84 %, identificaron 18 fallos

Cuadro XII
HERRAMIENTAS Y CONTROLES ORGANIZADOS POR FASE DEL PIPELINE

Fase	Objetivo de seguridad	Herramientas o controles representativos	Evidencia
Diseño	Definir requisitos, amenazas y propiedades verificables.	Modelado de amenazas, requisitos de seguridad y propiedades para monitoreo.	[6], [17], [19]
Build	Detectar defectos de código y dependencias antes de generar el artefacto.	SonarQube, Mega-Linter, Super-Linter, Snyk, SAST y SCA.	[16]–[18]
Predesplicue	Validar infraestructura, políticas, imágenes y funcionalidad.	Checkov, KICS, KubeLinter, Terrascan, OPA, Kyverno, análisis de Helm y security gates.	[8], [9], [12], [18]
Runtime	Evaluar el comportamiento real y detectar amenazas en ejecución.	OWASP ZAP, RESTest, EvoMaster, Schemathesis, RESTler, RestTestGen y detección de anomalías.	[7], [15], [17], [21]
Feedback	Priorizar, remediar y devolver hallazgos al equipo.	Gestión de findings, análisis de causa raíz, reanálisis y monitoreo continuo.	[9], [19], [20]

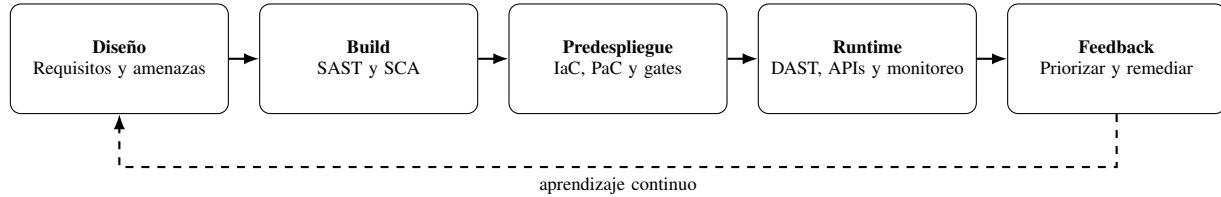


Figura 5. Distribución de controles de seguridad y ciclo de retroalimentación en el pipeline DevSecOps.

Cuadro XIII
PLATAFORMAS CI/CD IDENTIFICADAS EN EL CORPUS

Plataforma	Uso descrito	Evidencia
GitHub Actions	Pipelines asociados a repositorios, plugins y análisis de configuraciones.	[9], [14]
GitLab CI	Integración continua y ejecución de analizadores de seguridad.	[6], [14]
Azure Pipelines/DevOps	Evaluación del ecosistema de plugins y aislamiento de ejecuciones.	[14]
Jenkins y Bamboo	Pasarelas de seguridad y validación multiplataforma de políticas e IaC.	[8]
CircleCI	Plataforma incluida en el análisis comparativo de servicios CI.	[14]
Kubernetes y Tekton	Automatización nativa de nube asociada con commits y ramas.	[6]

Cuadro XIV
IMPACTOS OPERATIVOS REPORTADOS POR LA LITERATURA

Indicador	Resultado reportado	Evidencia
Revisión manual	Retraso de 25–40 minutos por configuración.	[8]
Remediación asistida	Reducción del 60 % en el tiempo de remediación.	[8]
Falsos positivos y cómputo	10 % de falsos positivos iniciales; ensamble con costo computacional tres veces mayor y latencia de 3.1 s en 500 casos IaC.	[8]
Pruebas REST sin hallazgos	Más del 70 % de las pruebas generadas no detectaron fallos.	[15]
Detección en ejecución	Latencia inferior a dos segundos en detección apoyada por ML.	[7]
Consistencia de analizadores	Resultados divergentes y necesidad de validación humana.	[9], [20]

potenciales y 23 regresiones; sin embargo, más del 70 % de las pruebas generadas no produjo hallazgos [15]. En GraphQL, el algoritmo MIO incrementó en 7 % la cobertura de líneas y encontró 10.2 fallos adicionales en promedio [21].

La validación de políticas IaC basada en modelos de lenguaje obtuvo un F1-score de 0.95 con una latencia de 3.1 segundos y redujo en 60 % el tiempo dedicado a revisión o remediación manual [8]. El pipeline de detección aumentado con aprendizaje automático reportó una tasa de detección de ataques de 95 % con una latencia inferior a dos segundos [7]. El estudio a gran escala de plugins de integración continua examinó la exposición potencial en 1,328,912 repositorios [14].

En el análisis de Helm, las herramientas mostraron diver-

gencias, falsos positivos y falsos negativos; las asignaciones excesivamente permisivas de ClusterRoles constituyeron la configuración errónea más recurrente [9]. El framework aumentado con IA registró tres casos de BOLA, dos de exposición excesiva de datos y uno relacionado con el control de acceso entre servicios [7].

IV-D3. Métricas propuestas y vacíos de medición: P01 compiló métricas como densidad de defectos, tasa de mitigación y perfilamiento de riesgos críticos, pero estas funcionaron como un marco de medición propuesto y no como valores derivados de una intervención experimental dentro del propio estudio [5]. VeriDevOps definió indicadores automáticos para artefactos, requisitos y monitoreo, aunque los extractos anali-

Cuadro XV
MÉTRICAS REPORTADAS Y NATURALEZA DE LA EVIDENCIA

Métrica	Contexto	Resultado o interpretación	Estado	Fuente
Cobertura	REST/GraphQL	Hasta 84 % en REST; incremento de 7 % en cobertura de líneas con MIO para GraphQL.	Medida	[15], [21]
Fallos y regresiones	Pruebas de APIs	18 fallos potenciales, 23 regresiones y 10.2 fallos adicionales en promedio con MIO.	Medida	[15], [21]
APFD y costo	Priorización de pruebas REST	Utilizadas para comparar detección temprana y costo; más del 70 % de las pruebas no produjo hallazgos.	Medida	[15]
F1, FPR y FNR	Validación de políticas IaC	F1=0.95; falsos positivos iniciales del 10 % y comparación de errores de clasificación.	Medida	[8]
Latencia	IaC y detección runtime	3.1 s en validación IaC y menos de 2 s en detección de amenazas con ML.	Medida	[7], [8]
Tasa de detección	Detección con ML	95 % de ataques detectados en el entorno experimental.	Medida	[7]
M_FOUND, M_BREAKING y M_REMAINING	Helm/IaC	Violaciones encontradas, mitigaciones que rompen funcionalidad y vulnerabilidades remanentes.	Medida	[9]
Densidad de defectos y burn rate	Marco DevSecOps	Métricas recopiladas como propuesta para evaluar calidad y mitigación.	Propuesta	[5]
MTTD y MTTR	Operación y observabilidad	No se encontraron valores empíricos comparables en el corpus.	Propuesta/no medida	[5], [10]
Perfil de riesgo crítico	Gestión de seguridad	Indicador sugerido para priorizar riesgos; sin evaluación before/after uniforme.	Propuesta	[5]

zados no proporcionaron valores numéricos comparables [19].

MTTD y MTTR no fueron medidos explícitamente con valores concretos en el corpus. La observabilidad y el monitoreo sí aparecieron como capacidades arquitectónicas recurrentes, pero no como series cuantitativas estandarizadas de detección o recuperación [10], [12], [24]. Por tanto, la evidencia actual permite comparar resultados técnicos locales, pero todavía no demuestra mediante indicadores uniformes el impacto organizacional de DevSecOps sobre la seguridad de APIs.

IV-E. Respuesta a la RQ4: vulnerabilidades reducidas

La evidencia distingue entre reducción medida, mejora operativa e identificación de vulnerabilidades. La reducción cuantificada se concentró en configuraciones de infraestructura y políticas; para los fallos funcionales de APIs predominó la detección, sin mediciones posteriores que demostraran cuánto disminuyeron después de aplicar un control.

IV-E1. Solidez de la evidencia de reducción: La evidencia más directa provino del pipeline para Helm charts, que detectó violaciones, aplicó mitigaciones predeterminadas, verificó que los cambios no rompieran la funcionalidad y midió el estado residual mediante M_REMAINING [9]. Este diseño permite afirmar una reducción de violaciones de política, aunque no produjo una tasa agregada única y mostró variabilidad, falsos positivos y falsos negativos entre analizadores.

La validación IaC con modelos de lenguaje aportó una mejora operativa: redujo 60 % el tiempo de revisión o remediación manual y 70 % el esfuerzo de mantenimiento, con F1=0.95 y latencia de 3.1 segundos [8]. Estos resultados no equivalen a una reducción medida de una categoría OWASP específica, pero sí muestran menor carga para validar y corregir configuraciones. La gestión industrial de hallazgos también mejoró el retorno de resultados al equipo, aunque sin valores antes/después por clase de vulnerabilidad [20].

En contraste, los estudios de APIs REST, GraphQL y micro-servicios identificaron BOLA, exposición excesiva de datos, regresiones y fallos de acceso, pero no midieron sistemáticamente su reducción después de implantar mitigaciones [7],

[12], [15], [21], [22]. El análisis de plugins de CI documentó aislamiento insuficiente y exposición de secretos o tokens a gran escala, sin una evaluación posterior de remediación efectiva de las plataformas [14].

IV-E2. Correspondencia con OWASP API Security Top 10:

Los hallazgos se mapearon con la edición 2023 de OWASP API Security Top 10 [2]. Los tres casos de BOLA correspondieron directamente a API1:2023 (*Broken Object Level Authorization*) [7]. Los dos casos de exposición excesiva de datos se relacionaron con API3:2023 (*Broken Object Property Level Authorization*), especialmente cuando la API devolvía propiedades sin filtrado suficiente [7].

Los fallos de acceso entre servicios, la escalación de privilegios y las identidades desalineadas se asociaron principalmente con API5:2023 (*Broken Function Level Authorization*) y, según el recurso afectado, con otras formas de autorización rota [7], [12], [13], [22]. Las configuraciones excesivamente permisivas en Helm y Kubernetes se vincularon con API8:2023 (*Security Misconfiguration*), aunque su origen inmediato perteneciera a IaC y a la orquestación [9].

La Figura 6 sintetiza la fuerza de la evidencia para cada relación entre vulnerabilidad y familia de mitigación. La escala distingue reducción medida (3), detección o identificación medida (2), mitigación propuesta sin evaluación posterior (1) y ausencia de evidencia directa en el corpus (0). En consecuencia, una celda con valor 2 no implica que la vulnerabilidad haya disminuido, sino únicamente que fue observada mediante una medición.

En síntesis, la reducción de vulnerabilidades está mejor demostrada para configuraciones y políticas automatizables. Para BOLA, BOPLA, autorización inter-servicios y fuga de secretos, el corpus respalda mecanismos de detección y mitigaciones plausibles, pero todavía carece de evaluaciones before/after que cuantifiquen la reducción por categoría.

Cuadro XVI
SEPARACIÓN EXPLÍCITA DE LOS TIPOS DE EVIDENCIA

Tipo de evidencia	Qué permite afirmar	Ejemplo y métrica asociada	Estudios
Reducción medida	Una intervención disminuyó violaciones y se evaluó el estado posterior.	Mitigación y reanálisis de Helm/IaC mediante M_FOUND, M_BREAKING y M_REMAINING.	[9]
Mejora operativa medida	El proceso requirió menos tiempo o esfuerzo, sin demostrar reducción de una categoría OWASP.	60 % menos revisión/remediación y 70 % menos mantenimiento; F1=0.95 y 3.1 s de latencia.	[8], [20]
Detección medida	Se localizaron vulnerabilidades o fallos, pero no se evaluó su reducción posterior.	Cobertura, faults, regresiones, tres BOLA, dos exposiciones de datos y un fallo de acceso.	[7], [15], [21]
Mitigación propuesta	La literatura relacionó un control con una amenaza sin una comparación before/after.	RBAC/ABAC, mTLS, OAuth 2.0, MFA, gateways, Vault y service mesh.	[12], [13], [22]–[24]

Cuadro XVII
RELACIÓN VULNERABILIDAD–MITIGACIÓN–EVIDENCIA

Vulnerabilidad	OWASP	Control o mitigación	Tipo de evidencia	Métrica o resultado	Fuente
BOLA/acceso a objetos	API1	Fuzzing, DAST, reglas de acceso, RBAC/ABAC, gateways y MFA.	Detección medida; mitigación propuesta.	Tres casos detectados; sin reducción postintervención.	[7], [12], [15]
Exposición de datos/BOPLA	API3	Generación de casos y validación del filtrado de respuestas.	Detección medida.	Dos casos detectados; sin medición posterior.	[7], [15]
Acceso indebido entre servicios	API5	mTLS, OAuth 2.0, RBAC/ABAC, XACML y service mesh.	Mitigación propuesta y mapeo arquitectónico.	Sin tasa uniforme de reducción.	[12], [13], [17], [24]
Configuración incorrecta	API8	Policy as Code, análisis de Helm/IaC, mitigación y reanálisis.	Reducción medida.	M_FOUND, M_BREAKING y M_REMAINING.	[9]
Fuga de secretos y tokens	API2 / pipeline	Vault, rotación, MFA y aislamiento de ejecución.	Identificación y mitigación propuesta.	Exposición a gran escala; sin before/after.	[13], [14], [23]

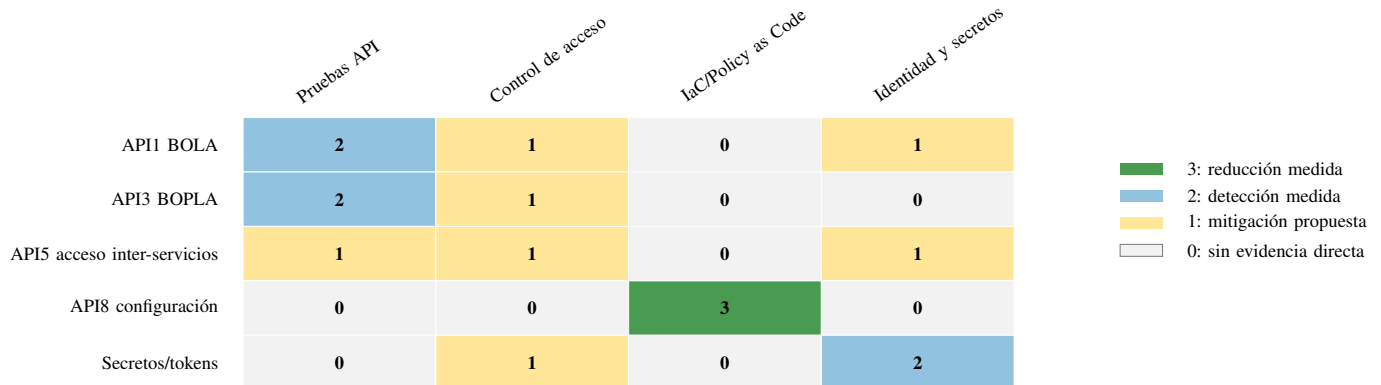


Figura 6. Mapa de calor de vulnerabilidades, familias de mitigación y fuerza de la evidencia.

V. DISCUSIÓN

V-A. Interpretación de los hallazgos

La seguridad automatizada no se reduce a seleccionar un escáner aislado, sino que requiere distribuir controles complementarios durante el ciclo de vida [7], [8], [16], [22]. En diseño y construcción predominan SAST, SCA y la validación de IaC o políticas; en pruebas se incorporan DAST, fuzzing y frameworks específicos para REST o GraphQL; finalmente, en despliegue y operación intervienen el escaneo de contenedores, la gestión de secretos, las mallas de servicios y la observabilidad [9], [10], [15], [18], [21], [23].

La amplitud de herramientas identificadas evidencia un ecosistema fragmentado. Motores como OPA o Kyverno y analizadores como Checkov, KICS, KubeLint y Terrascan facilitan el tratamiento de políticas e infraestructura como

código, pero la evidencia disponible no permite declarar una solución universalmente superior [8], [9], [12], [16], [18]. La selección depende del tipo de API, la plataforma de despliegue, el nivel de acceso al código y la madurez del pipeline.

En cuanto a vulnerabilidades, la literatura distingue los fallos propios de la API de los riesgos de la infraestructura que la soporta. BOLA, la exposición excesiva de datos y los defectos de autenticación o autorización pertenecen al plano de la lógica y la identidad de la API [2], [12], [22]. En contraste, los complementos inseguros de CI/CD constituyen un riesgo de la cadena de suministro [14], mientras que permisos excesivos o políticas defectuosas en Helm y Kubernetes corresponden al plano de orquestación [9]. Esta separación es necesaria para atribuir correctamente el efecto de cada control.

La medición cuantitativa continúa siendo menos madura que

la automatización técnica. Se reportan cobertura, fallos detectados, regresiones, costo de ejecución, densidad de defectos y tasas de mitigación [5], [15], [21]; sin embargo, no existe un conjunto uniforme de indicadores que permita comparar directamente todos los estudios. Métricas operativas como MTTD, MTTR o cumplimiento de SLA aportan información sobre la operación, pero no equivalen por sí mismas a una medida de seguridad intrínseca de la API [5], [10], [11].

V-B. Implicaciones para la práctica

Los resultados sugieren estructurar el pipeline mediante controles escalonados. Las verificaciones rápidas y deterministas, como linters, SAST, SCA y validación de IaC, pueden ejecutarse durante los cambios de código; las pruebas DAST y de APIs requieren un entorno desplegado; y la observabilidad, la gestión de secretos y el cumplimiento de políticas deben mantenerse durante la operación. Esta distribución permite detectar problemas cerca de su origen sin concentrar todas las verificaciones al final del ciclo.

La automatización también requiere mecanismos de priorización. Los falsos positivos, la heterogeneidad de resultados y la acumulación de hallazgos pueden convertir los escáneres en un cuello de botella si no existen reglas de severidad, responsables y criterios para bloquear o permitir un despliegue [9], [20]. Por ello, la decisión no debe basarse solamente en la cantidad de herramientas, sino en su integración con el flujo de trabajo y en la utilidad de la evidencia que producen.

V-C. Limitaciones y agenda de investigación

Persisten barreras relacionadas con la selección y mantenimiento de herramientas, la integración con sistemas legados, la actualización frente a nuevas amenazas, la capacitación y la coordinación entre desarrollo, operaciones y seguridad [6], [16]. En microservicios se suman problemas de identidad entre servicios, autenticación mutua, correlación de observabilidad y contención de ataques distribuidos [22]–[24].

Se requieren evaluaciones comparativas reproducibles que midan simultáneamente vulnerabilidades detectadas, falsos positivos, tiempo de remediación, costo computacional y efecto sobre la frecuencia de despliegue. También resulta necesario estudiar qué combinaciones de controles ofrecen una cobertura suficiente para APIs REST y GraphQL, y establecer métricas comunes que permitan comparar herramientas, arquitecturas y contextos organizacionales.

VI. AMENAZAS A LA VALIDEZ

Las amenazas se analizaron en cuatro dimensiones. La Tabla XVIII distingue el posible impacto, las medidas aplicadas durante esta revisión y las mejoras recomendadas para futuras actualizaciones.

En conjunto, estas amenazas no invalidan la síntesis, pero limitan la fuerza causal y el alcance de sus conclusiones. La trazabilidad del protocolo permite identificar qué afirmaciones provienen de mediciones directas y cuáles deben interpretarse como recomendaciones o evidencia contextual.

VII. CONCLUSIONES

Esta revisión de la literatura sistemática analizó 24 fuentes sobre la integración de seguridad de APIs en pipelines DevOps/DevSecOps. El propósito fue identificar las prácticas y herramientas utilizadas, describir su incorporación en los workflows, examinar las métricas reportadas y determinar qué vulnerabilidades presentan evidencia de reducción.

Los hallazgos muestran una estrategia por capas: SAST y SCA durante la construcción, validación de IaC y políticas antes del despliegue, DAST y pruebas específicas de APIs en entornos ejecutables, y monitoreo y controles de plataforma durante la operación. La automatización puede disminuir el trabajo manual y acelerar la detección, pero también introduce sobrecarga, falsos positivos y necesidades de priorización.

La evidencia más sólida de reducción se encontró en configuraciones Helm/IaC y validación de políticas. Para BOLA, BOPLA, autorización entre servicios y exposición de secretos predominan la identificación y las mitigaciones propuestas, sin comparaciones cuantitativas consistentes después de su implementación. Por ello, se requieren estudios reproducibles con métricas comunes, evaluaciones before/after y mediciones simultáneas de seguridad, costo computacional y velocidad de despliegue.

CONVENCIÓN DE LAS FUENTES

Las referencias [1]–[4] corresponden a literatura gris (LG01–LG04), las referencias [5]–[21] a estudios recuperados mediante la búsqueda primaria (P01–P17) y las referencias [22]–[24] a estudios incorporados mediante *snowballing* (SB1–SB3). La referencia [25] corresponde a PRISMA 2020 y no forma parte del corpus analizado.

REFERENCIAS

- [1] OWASP Foundation, “OWASP DevSecOps Guideline,” OWASP, s. f., consultado: 17 de julio de 2026. [Online]. Available: <https://owasp.org/www-project-devsecops-guideline/>
- [2] —, “OWASP Top 10 API Security Risks—2023,” OWASP API Security Project, 2023, consultado: 17 de julio de 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- [3] —, “Introducción: Top 10 controles proactivos de OWASP,” OWASP Top 10 Proactive Controls, s. f., consultado: 17 de julio de 2026. [Online]. Available: <https://top10proactive.owasp.org/the-top-10/>
- [4] Salt Security, “API Security Trends: API Attacks, Breaches & Threats Report,” Salt Security, s. f., consultado: 17 de julio de 2026. [Online]. Available: <https://salt.security/api-security-trends>
- [5] J. Y. Zhang and Y. Zhang, “Quantitative DevSecOps metrics for cloud-based web microservices,” *IEEE Access*, vol. 12, pp. 160 317–160 342, 2024.
- [6] S. Nagasundari, P. Manja, P. Mathur, and P. B. Honnavalli, “Extensive review of threat models for DevSecOps,” *IEEE Access*, vol. 13, pp. 45 252–45 271, 2025.
- [7] A. Mittal, “AI-augmented DevSecOps pipelines for secure and scalable service-oriented architectures in cloud-native systems,” in *2025 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, 2025, pp. 79–84.
- [8] A. Mittal and V. Venkatesan, “Practical integration of large language models into enterprise CI/CD pipelines for security policy validation: An industry-focused evaluation,” in *2025 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, 2025, pp. 197–203.
- [9] F. Minna, A. Blaise, K. Tuma, and F. Massacci, “Automated analysis of security policy violations in Helm charts,” *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 2, pp. 2519–2533, 2026.

Cuadro XVIII
AMENAZAS A LA VALIDEZ Y ACCIONES DE MITIGACIÓN

Amenaza	Impacto posible	Mitigación aplicada	Mejora futura
Constructo: vocabulario e indexación	Omisión de estudios que utilicen términos distintos para APIs, seguridad o CI/CD.	Cadena derivada de PICOC, sinónimos en tres bloques, adaptación conservando operadores, literatura gris y backward snowballing.	Pilotear la cadena con un conjunto de control, añadir nuevos sinónimos y validar sensibilidad por base de datos.
Interna: un único revisor	Sesgo de selección, extracción o interpretación y ausencia de medición del acuerdo.	Criterios previos, PRISMA, identificadores estables, formulario uniforme y comprobación auxiliar con Consensus contra las fuentes.	Incorporar al menos un segundo revisor, resolver desacuerdos y calcular acuerdo interevaluador.
Interna: calidad de los estudios	Inclusión de evidencia con diferente rigor y riesgo de sesgo.	Control de elegibilidad por tipo de fuente, revisión por pares, pertinencia y texto completo; separación entre evidencia medida y propuesta.	Aplicar una herramienta formal de evaluación de calidad o riesgo de sesgo por diseño de estudio.
Externa: concentración cloud-native	Generalización inadecuada a APIs monolíticas, locales o sin CI/CD.	Alcance limitado explícitamente a APIs cloud-native, microservicios, contenedores y pipelines CI/CD.	Comparar con APIs monolíticas, arquitecturas no cloud y organizaciones con distintos niveles de madurez.
Disponibilidad: acceso manual	Sesgo por los cinco textos no recuperados y mayor probabilidad de error durante la recolección.	Registro del flujo PRISMA, recuperación individual y lectura completa de las 24 fuentes incluidas.	Utilizar acceso institucional, contactar autores y archivar anticipadamente los textos y metadatos.
Conclusiones: heterogeneidad	Comparaciones débiles entre herramientas, métricas y contextos; imposibilidad de metaanálisis.	Síntesis narrativa por RQ y distinción entre reducción medida, mejora operativa, detección y mitigación propuesta.	Definir métricas comunes, promover estudios before/after y realizar metaanálisis cuando exista evidencia comparable.

- [10] U. Faseeha, H. J. Syed, F. Samad, S. Zehra, and H. Ahmed, “Observability in microservices: An in-depth exploration of frameworks, challenges, and deployment paradigms,” *IEEE Access*, vol. 13, pp. 72 011–72 039, 2025.
- [11] P. P. Patharlagadda, “Business-aware SLA-driven autoscaling for Kubernetes microservices using application-level observability,” *IEEE Access*, vol. 14, pp. 66 554–66 564, 2026.
- [12] S. M. Saleh, M. N. A. Ifat, N. H. Madhavji, and J. Steinbacher, “A defence-oriented study of API security in CI/CD pipelines,” in *Proceedings of the 2025 10th International Conference on Cloud Computing and Internet of Things*. ACM, 2025, pp. 33–42.
- [13] S. M. Saleh, N. H. Madhavji, and J. Steinbacher, “Systematic review of identity-centric security in cloud-native CI/CD pipelines,” in *Proceedings of the 2025 10th International Conference on Cloud Computing and Internet of Things*. ACM, 2025, pp. 23–32.
- [14] X. Li, Y. Gu, C. Qiao, Z. Zhang, D. Liu, L. Ying, H. Duan, and X. Gao, “Toward understanding the security of plugins in continuous integration services,” in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2024, pp. 482–496.
- [15] H. Sartaj, S. Ali, and J. Marie Gjøby, “REST API testing in DevOps: A study on an evolving healthcare IoT application,” *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 6, 2026.
- [16] J. A. Guyong, R. M. Balaybay, J. M. Macario, E. J. Pangan, and J. R. Clemente, “Implementing DevSecOps: A systematic literature review on integrating security into DevOps,” in *Proceedings of the 2025 7th World Symposium on Software Engineering*. ACM, 2025, pp. 42–47.
- [17] L. Verderame, L. Caviglione, R. Carbone, and A. Merlo, “SecCo: Automated services to secure containers in the DevOps paradigm,” in *Proceedings of the International Conference on Research in Adaptive and Convergent Systems*. ACM, 2023.
- [18] M. Cankar, N. Petrovic, J. Pita Costa, A. Cernivec, J. Antic, T. Martincic, and D. Stepec, “Security in DevSecOps: Applying tools and machine learning to verification and monitoring steps,” in *Companion of the 2023 ACM/SPEC International Conference on Performance Engineering*. ACM, 2023, pp. 201–205.
- [19] E. P. Enoiu, D. Truscan, A. Sadovykh, and W. Mallouli, “VeriDevOps software methodology: Security verification and validation for DevOps practices,” in *Proceedings of the 18th International Conference on Availability, Reliability and Security*. ACM, 2023.
- [20] M. Voggenreiter, F. Angermeir, F. Moyon, U. Schöpp, and P. Bonvin, “Automated security findings management: A case study in industrial DevOps,” in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*. ACM, 2024, pp. 312–322.
- [21] A. Belhadi, M. Zhang, and A. Arcuri, “Random testing and evolutionary testing for fuzzing GraphQL APIs,” *ACM Transactions on the Web*, vol. 18, no. 1, 2024.
- [22] P. Haindl, P. Kochberger, and M. Svegggen, “A systematic literature review of inter-service security threats and mitigation strategies in microservice architectures,” *IEEE Access*, pp. 90 252–90 286, 2024.
- [23] P. Billawa, A. Bambhore Tukaram, N. E. Díaz Ferreyra, J.-P. Steghöfer, R. Scandariato, and G. Simhandl, “SoK: Security of microservice applications: A practitioners’ perspective on challenges and best practices,” in *Proceedings of the 17th International Conference on Availability, Reliability and Security*. ACM, 2022, pp. 1–10.
- [24] C. Meadows, S. Hounsinnou, T. Wood, and G. Bloom, “Sidecar-based path-aware security for microservices,” in *Proceedings of the 28th ACM Symposium on Access Control Models and Technologies*. ACM, 2023, pp. 157–162.
- [25] M. J. Page, J. E. McKenzie, P. M. Bossuyt *et al.*, “The PRISMA 2020 statement: An updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, p. n71, 2021.